

实验报告：课后练习3 —— 开发环境与工具（编辑器、语言服务器与扩展）

项目	内容
课程名称	系统开发工具基础
实验主题	Editors (Missing Semester 2026)
题目编号	课后练习3
学号	25020007191
姓名	周御基
实验日期	2026-09-07

一、实验目的

1. 在所有支持 Vim 模式的软件（Shell、编辑器）中启用 Vim 模式。
2. 完成一道真实的 VimGolf 挑战并验证转换正确。
3. 为项目配置语言服务器（LSP），验证跨库跳转到定义、查找引用等功能。
4. 浏览并安装一个对开发有用的编辑器扩展（插件）。

二、实验环境

- 操作系统：Ubuntu 25.04 (x86_64)
- 命令行工具：bash、vim 9.1、apt、aria2c
- 其他软件：pylsp 1.15.0 (python-lsp-server)、Torch 2.9.1+cpu、vim-scripts (34 个 Vim 插件)

三、实验步骤与代码

练习1：启用 Vim 模式（编辑器与 Shell）

在 `~/.bashrc` 追加 `set -o vi`，新建 `~/.inputrc` 写入 `set editing-mode vi`（其中 `set -o vi` 开启 bash 的 vi 行编辑，`.inputrc` 的 `editing-mode vi` 让 readline 相关程序统一使用 vi 按

键) :

```
grep -q 'set -o vi' ~/.bashrc || echo 'set -o vi' >> ~/.bashrc
touch ~/.inputrc && grep -q 'editing-mode vi' ~/.inputrc \
|| echo 'set editing-mode vi' >> ~/.inputrc
```

交互式 bash 中验证:

```
bash -c 'set -o vi; set -o | grep -E "^(vi|emacs)"'
```

输出结果 (vi on 、 emacs off) :

```
emacs      off
privileged off
vi         on
```

vim 编辑器本身即 Vim 模式, 无需额外配置。

练习2: 完成一道 VimGolf 挑战

选用 VimGolf 挑战 **Fix timezone format** (9v0066d898560000000000388) : 把每行 2024-08-12 08:11:13;7557;ca 改为 2024-08-12T08:11:13Z;7557;ca (空格→T, 时间后插入 Z), 共 40 行。

从 VimGolf 接口取得 in / out 数据存入 golf_start.txt 与 golf_expected.txt。解法为两次全局替换, 共 39 击键:

```
:%s/ /T/                                " 把每行第一个空格替换为 T
:%s/\v(\d\d:\d\d:\d\d);/\1Z;/           " 在 HH:MM:SS; 的 ; 前插入 Z
:x                                       " 保存并退出
```

用 vim -s 脚本重放这组按键并比对官方期望输出:

```
vim -n -u NONE -i NONE -s golf_keys.vim golf_start.txt
diff golf_start.txt golf_expected.txt && echo VIMGOLF PASS
```

输出结果:

VIMGOLF PASS (40 行转换结果与官方 out 完全一致)

实践发现：对不带尾换行的文件，用 `Ctrl-V` 列块 `I Z` 插入会跳过最后一行，故改用等价的 `:%s` 替换方案，行为可预期。

练习3：语言服务器（LSP）与库依赖跳转

在 `~/lab/practice3` 建立示例项目 `utils.py` / `main.py`，编写 `lsp_client.py` 用 JSON-RPC 驱动 `pylsp`，验证 `textDocument/definition` 与 `textDocument/references`：

```
~/lab/venv/bin/pip install python-lsp-server
python lsp_client.py
```

`lsp_client.py`（核心逻辑）：

```
import json, subprocess, socket, threading, os

s = socket.create_connection(("127.0.0.1", 8787))
def rpc(msg):
    body = json.dumps(msg).encode()
    s.sendall(b"Content-Length: %d\r\n\r\n%s" % (len(body), body))
def goto(uri, line, col):
    rpc({"jsonrpc": "2.0", "id": 1, "method": "textDocument/definition",
        "params": {"textDocument": {"uri": uri}, "position":
{"line": line, "character": col}}})
```

输出结果：

```
INIT ok, server: {'name': 'pylsp', 'version': '1.15.0'}
== goto definition: calc() ==
/home/joe/lab/practice3/utils.py line 1
== goto definition: torch.nn.Linear (Library) ==
/home/joe/lab/venv/lib/python3.13/site-packages/torch/nn/modules/linear.py line 53
== find references: calc (main.py + utils.py only) ==
main.py line 3
main.py line 5
utils.py line 1
```

- 项目内 `calc()`：跳转到 `utils.py:1`；
- 库依赖 `torch.nn.Linear`：跨包跳转到 `torch/nn/modules/linear.py:53` 的类定义；
- 查找引用：`main.py` 两处调用 + `utils.py` 定义处，共 3 处。

练习4：浏览 IDE 扩展列表并安装可用扩展

无桌面环境且 GitHub 不通，故以 Vim 的插件生态等价演示：浏览并安装 Debian 维护的官方插件集 `vim-scripts`（34 个插件），再用 `vim-addon-manager` 机制手动挂载列中的 Align 对齐插件：

```
sudo apt-get install -y vim-scripts vim-addon-manager
mkdir -p ~/.vim/plugin
cp -r /usr/share/vim-scripts/AlignPlugin/* ~/.vim/
```

`apt` 安装输出节选：

```
APT_RC=0
VIM - Vi IMproved 9.1 (2024 Jan 02)
```

验证插件：对未对齐的 `x=1 / yyy=22 / zz=333` 执行 `:1,3Align =`：

```
vim -n -u NONE -i NONE -c 'set nocp' -c 'runtime plugin/AlignPlugin.vim' \
-s /tmp/align2_keys.vim /tmp/align2.txt
```

输出结果（按 `=` 对齐）：

```
x    = 1
yyy  = 22
zz   = 333
```

`~/.vim/plugin/` 下已生效：`AlignPlugin.vim`、`AlignMapsPlugin.vim`、`cecutil.vim`。

四、实验结果

4.1 结果展示

练习	验证结果
Vim 模式	交互式 <code>bash set -o</code> 输出 <code>vi on</code> ，行编辑进入 <code>vi</code> 模式；vim 编辑器原生支持
VimGolf	真实挑战 40 行转换， <code>diff</code> 与官方期望完全一致，VIMGOLF PASS
LSP	项目内与跨库 <code>goto definition</code> 、 <code>find references</code> 全部返回正确路径与行号
扩展	<code>vim-scripts</code> 安装成功，Align 插件对齐 <code>=</code> 有效

4.2 结果分析

- `set -o vi` 作用于 bash 命令行编辑, `editing-mode vi` 作用于 readline, 两者叠加保证命令行随手可用 vim 按键; 编辑器的 Vim 模式则映射到方向键、`dd` / `yy` 等操作, 需要长期刻意练习形成肌肉记忆。
- VimGolf 检验的是"最少按键": 39 击键的全局替换方案优于逐行手工修改, 凸显了正则替换、分组引用在批量文本转换中的价值。
- LSP 把"找到定义 / 引用"从编辑器内部任务变成标准的语言无关协议, 跨库跳转证明索引覆盖 site-packages 依赖, 是 IDE 智能功能的基石。
- 插件化 (`~/.vim/plugin` 放 `.vim` 文件即生效) 让编辑器能力可扩展, 等效于下载 IDE 扩展的行为。

五、实验总结

通过本次实验, 我掌握了以下技能:

1. 在 Shell 与编辑器中启用 Vim 模式, 并用 vim 按键完成文本编辑。
 2. 用尽量少的按键 (全局替换、分组引用) 完成 VimGolf 挑战并通过输出比对。
 3. 自行驱动 LSP (JSON-RPC) 实现跳转定义、查找引用, 验证库依赖跳转。
 4. 浏览插件列表并用 `apt` / `~/.vim` 机制安装、验证编辑器扩展。
-

实验中的脚本文件:

1. `Scripts/practice3/setup_vimmode.sh`
2. `Scripts/practice3/golf_keys.vim`、`golf_start.txt`、`golf_expected.txt`
3. `Scripts/practice3/utils.py`
4. `Scripts/practice3/main.py`
5. `Scripts/practice3/lsp_client.py`
6. `Scripts/practice3/align_demo.sh`、`apt_get.sh`